SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE CODE: CSA51 COURSE NAME: Cryptography and Network Security

COURSE OUTCOME COVERED

CO3: Examine cryptographic hash algorithms, digital signature schemes, and assess Kerberos and X.509 authentication frameworks for ensuring integrity, authentication, and non-repudiation. CO (BL4, BL5)

Assessment Tool Weightage: 20%

QUESTIONS: 5 TOTAL MARKS: 25

CASE STUDY

Code of Conduct

I T Sanjeeva Reddy (Name / Reg No: 192372304) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Name of the student: T Sanjeeva Reddy

Registration Number: 192372304

Signature of the student: T. Sanjeeva Reddy

Question 5 — X.509 Certificate Trust Validation Errors

Scenario

An organization implements X.509 certificates for secure communication, but users encounter frequent trust validation errors. Analyze the possible issues in certificate management and propose solutions to ensure proper authentication and trust chain validation.

Analysis of Possible Issues

- Broken chain of trust: an intermediate CA certificate is missing from the server's certificate bundle, so the client cannot build a complete path from the leaf certificate up to a trusted root, even though the root itself is trusted.
- Expired or not-yet-valid certificates: certificates carry validity periods (notBefore / notAfter); an expired leaf or intermediate certificate, or a system clock that is out of sync, causes validation failures.
- Untrusted or self-signed root: internal systems sometimes use a self-signed or private CA whose root certificate was never distributed to client trust stores, so every client rejects the chain.
- Hostname / Subject mismatch: the Common Name or Subject Alternative Name (SAN) in the certificate does not match the hostname being contacted, which browsers and TLS libraries treat as a hard failure.
- Revoked certificates not detected: if CRL (Certificate Revocation List) or OCSP (Online Certificate Status Protocol) checking is disabled or the responder is unreachable, a compromised or revoked certificate may be silently accepted, or conversely a temporary OCSP outage may cause valid certificates to be rejected.
- Weak or deprecated signature algorithms (e.g., SHA-1-signed certificates) are rejected by modern validators, producing trust errors even though the certificate content itself is otherwise correct.
- Poor certificate lifecycle management: no centralized inventory of certificate expiry dates leads to certificates lapsing unnoticed until users start seeing errors.

Proposed Solutions

- Always deploy the full certificate chain (leaf + all intermediates) on the server, and periodically validate the chain with tools such as openssl s_client or an automated chain-checker.
- Use a well-known public CA (or, for internal systems, distribute the private root CA certificate to all client trust stores through group policy / MDM) so the root is properly trusted.
- Implement automated certificate lifecycle management (e.g., ACME protocol with Let's Encrypt, or an enterprise certificate manager) to auto-renew certificates well before expiry and alert administrators of upcoming expirations.
- Enforce correct SAN entries covering every hostname/IP the certificate will be presented for, and validate this as part of the certificate issuance workflow.
- Enable and monitor OCSP stapling, which lets the server proactively provide revocation status to clients, improving both reliability and privacy compared with

client-side OCSP/CRL lookups.

- Require modern signature algorithms (SHA-256 or stronger) and current TLS versions (TLS 1.2/1.3) when issuing or renewing certificates.
- Synchronize system clocks (NTP) across servers and clients, since even a few minutes of clock drift can trigger notBefore/notAfter validation failures.
- Maintain a centralized certificate inventory/dashboard so expiring or misconfigured certificates are caught before they cause outages.

Conclusion

Most X.509 trust validation errors stem from incomplete chains, expired certificates, hostname mismatches, or an undistributed root of trust rather than a flaw in X.509 itself. Systematic certificate lifecycle management, automated renewal, complete chain deployment, and OCSP stapling together restore reliable authentication and trust chain validation.

Question 7 — Symmetric Key Management After Employee Turnover

Scenario

An e-commerce application uses a symmetric key encryption system to protect customer payment information. After several employees leave the organization, the company discovers that former employees may still have access to encryption keys. Analyze the key management problems in the system and propose a secure key generation, distribution, storage, and revocation mechanism.

Analysis of Key Management Problems

- Static, long-lived shared keys: if the same symmetric key has been used since deployment without rotation, anyone who ever had access to it — including former employees — can still decrypt payment data indefinitely.
- No individual accountability: symmetric keys are typically shared among a team, so there is no way to trace which individual used the key or to revoke access for just one person without disrupting everyone else.
- Keys stored insecurely: if keys were embedded in configuration files, source code, or shared documents/spreadsheets rather than a dedicated key vault, departing employees may have copied or memorized them, or retain access to the storage location.
- No revocation process tied to offboarding: the organization's employee-exit checklist apparently did not include revoking or rotating cryptographic key access, which is a process failure, not just a technical one.
- No key versioning: without support for multiple concurrent key versions, rotating the key means all already-encrypted data becomes unreadable unless a re-encryption/migration plan exists.

Proposed Secure Key Management Mechanism

1. Key Generation

- Generate symmetric keys (e.g., AES-256) using a cryptographically secure random number generator (CSPRNG), never derived from predictable inputs such as passwords typed by staff.
- Generate keys inside a Hardware Security Module (HSM) or a managed cloud Key Management Service (KMS) such as AWS KMS, Azure Key Vault, or Google Cloud KMS, so the raw key material never needs to leave the secure boundary.

2. Key Distribution

- Never distribute raw keys to individual employees. Instead, applications should request encryption/decryption operations from the KMS/HSM using scoped, audited API calls — the employee never sees the key itself.
- Where key material must be shared between systems, use key-wrapping: encrypt the symmetric key with a public key (asymmetric envelope encryption) so it can be transmitted safely and only unwrapped by the intended service.

3. Key Storage

- Store keys exclusively in an HSM or KMS with strict access-control policies (role-based access control, least privilege), never in application code, config files, or plaintext documents.
- Encrypt data using envelope encryption: a per-record or per-batch data key encrypts the payment data, and that data key is itself encrypted by a master key held in the KMS/HSM — this limits the blast radius of any single key exposure.

4. Key Rotation and Revocation

- Rotate symmetric keys on a defined schedule (e.g., every 90 days) and immediately upon any employee's role change or departure that involved privileged access.
- Tie key/access revocation directly into the HR offboarding workflow, so IT security automatically revokes an employee's KMS access and, where warranted, triggers key rotation the moment their employment ends.
- Maintain multiple key versions so newly encrypted data uses the current key while older data can still be decrypted with archived (but access-controlled) previous key versions until it is re-encrypted.
- Log every key-usage and access-control event (who requested what operation, when) through the KMS/HSM's audit trail, enabling accountability and forensic review — something impossible with a shared static key.

Conclusion

The core failure here is treating a symmetric key as a static shared secret rather than a managed, auditable resource. Centralizing key generation and storage in an HSM/KMS, distributing only scoped operations rather than raw key material, enforcing role-based access, and linking key rotation/revocation to the employee offboarding process together close the gap that allowed former employees to retain access.